

Explainability

- From Attribution to Language Models -

Department of Data Science

Ewha Womans University

Prof. Sangyeop Kim



이화여자대학교
EWA WOMANS UNIVERSITY



Why We Explain a Model

A model that predicts well still has to explain its predictions.

Who asks	What they need
Regulators	A reason for each loan decision
Portfolio managers	Why a signal fired before they trade on it
The modeler	A way to catch mistakes, such as a leaked column

An accuracy score tells us how often the model is right. An **explanation** tells us why it gave this particular answer.



Two Kinds of Input

We first explain models built on tables, then models that read text.

	Tables	Text
Example	Daily prices and indicators	News, filings, posts
One observation	One firm on one day	One sentence or document
Inputs	Fixed columns such as volume	Words, and their number varies
An explanation says	"Volume raised the forecast"	"'reversed' made it positive"

We ask the same question of both: which part of the input moved the prediction?



What an Explanation Can Tell Us

An explanation can tell us four different things about how a model uses its features.

What it tells us	Example	Methods
How much the model relies on a feature	Volume ranks first, tone last	Permutation, tree importance
Which way, and by how much	One more unit of volume adds 0.4	Linear coefficients, partial dependence
One prediction, split by addition	$0.30 + 0.10 + 0.06 = 0.46$	SHAP
An interaction between features	Volume raises the forecast only when tone is negative	Tree splits, two-way partial dependence

A breakdown adds up exactly, so an interaction has to be split between the features involved.



Global and Local Explanation

Some explanations describe the whole model, and others describe a single prediction.

Global: the whole model		Local: one prediction
Intrinsic: the model is readable	Linear coefficients, shallow trees	The path one row takes in a tree
Post-hoc: computed after training	Permutation importance, partial dependence	SHAP

- Interpretability: we can read the model directly.
- Explainability: a separate method describes the model, and that method can be wrong.

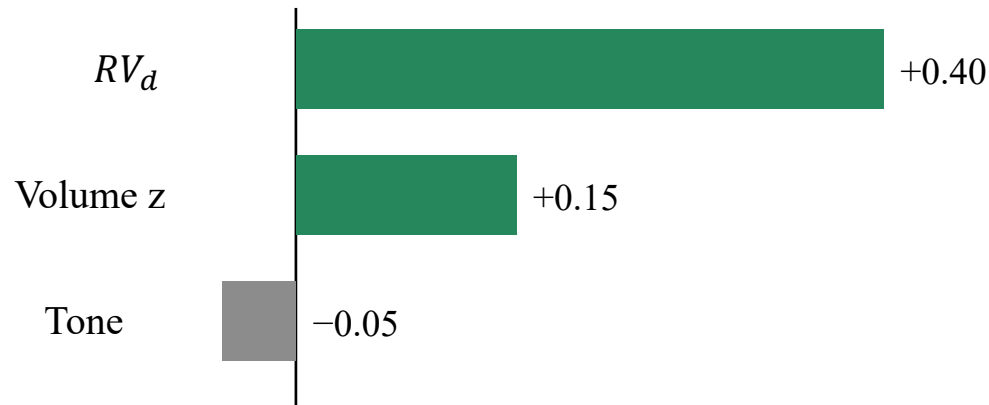
SHAP gives both the global and the local view from one calculation.



Explaining a Linear Model

In a linear model, each coefficient says how much the prediction moves when its feature rises by one.

$$\text{Predicted volatility} = 0.50 + 0.40 \times RV_d + 0.15 \times \text{Volume } z - 0.05 \times \text{Tone}$$

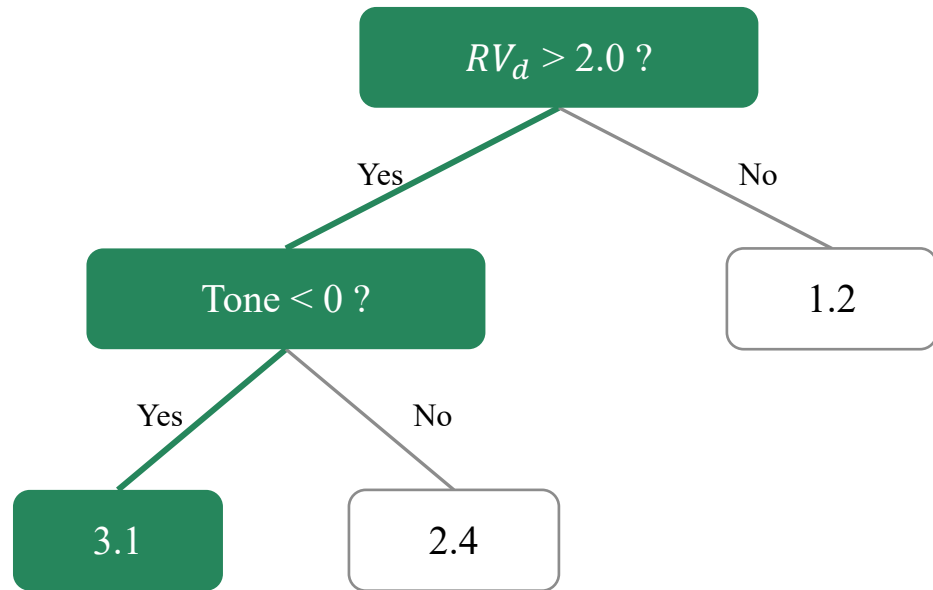


- Sign gives the direction: a higher tone lowers the forecast.
- Size depends on units, so standardize before comparing.
- Correlated inputs share one effect, so coefficients are unstable.

A linear model is easy to read, but it can only describe a straight-line relationship.

Explaining a Decision Tree

A tree explains a prediction by the path the row took through the splits.



- This row: $RV_d = 2.6$, $Tone = -0.3$.
- Its path: $RV_d > 2.0$, then $Tone < 0$, so the tree predicts 3.1.
- The split at the top is the feature the tree uses first.
- Depth two can be read by eye; depth twenty cannot.

Here the explanation is the model itself, so it is exact.



Feature Importance in a Random Forest

A forest has hundreds of trees, so we add up how much each feature's splits helped.

Feature	Tree 1	Tree 2	Tree 3	Total	Share
RV_d	0.30	0.25	0.20	0.75	60%
Volume z	0.10	0.15	0.05	0.30	24%
Tone	0.05	0.00	0.15	0.20	16%

- Each number is how much that feature's splits reduced impurity in one tree.
- The total is divided by the grand total, 1.25, to give each feature's share.
- Continuous and many-category features get more splits and tend to score higher.

Tree importance shows which features the model relies on, but not which way they push the prediction.



What a Shapley Value Is

Lundberg and Lee (2017), A Unified Approach to Interpreting Model Predictions, NeurIPS. Approx. 30,000 citations.

- The game: features are players, and the payoff is the prediction minus the average prediction.
- The rule: a feature's credit is what it adds, averaged over every order of arrival.
- Why this rule: it is the attribution that satisfies efficiency, symmetry and dummy.



The Set-Up and the Six Orderings

Take one firm-day with $D = RV_d$, $W = RV_w$ and $S = \text{sentiment}$; v is the prediction minus 0.30.

Revealed	none	D	W	S	D, W	D, S	W, S	D, W, S
Prediction	0.30	0.40	0.36	0.32	0.43	0.43	0.39	0.46
v	0.00	0.10	0.06	0.02	0.13	0.13	0.09	0.16

Ordering	When D arrives	Marginal contribution of D	Value
D, W, S and D, S, W	first	$v(D) - v(\text{none}) = 0.10 - 0.00$	0.10, twice
W, D, S	second, after W	$v(D, W) - v(W) = 0.13 - 0.06$	0.07
S, D, W	second, after S	$v(D, S) - v(S) = 0.13 - 0.02$	0.11
W, S, D and S, W, D	last	$v(D, W, S) - v(W, S) = 0.16 - 0.09$	0.07, twice

$$\phi_D = (0.10 + 0.10 + 0.07 + 0.11 + 0.07 + 0.07) / 6 = 0.52 / 6 = 0.0867$$



Three Attributions and the Efficiency Check

The same averaging for W and S gives the other two attributions.

Feature	Sum over the six orderings	$\phi = \text{sum} / 6$	Interpretation
RV_d	$0.10 + 0.10 + 0.07 + 0.11 + 0.07 + 0.07 = 0.52$	+0.0867	Raises the prediction most
RV_w	$0.06 + 0.06 + 0.03 + 0.07 + 0.03 + 0.03 = 0.28$	+0.0467	Second, overlaps with RV_d
Sentiment	$0.02 + 0.02 + 0.03 + 0.03 + 0.03 + 0.03 = 0.16$	+0.0267	Smallest, but not zero

$$0.0867 + 0.0467 + 0.0267 = 0.1600 = v(D, W, S), \text{ and } 0.30 + 0.1600 = 0.46$$

- The three attributions add up exactly to the change from the baseline.
- Sentiment gets a sixth of the change, yet permutation importance will rate it near zero.

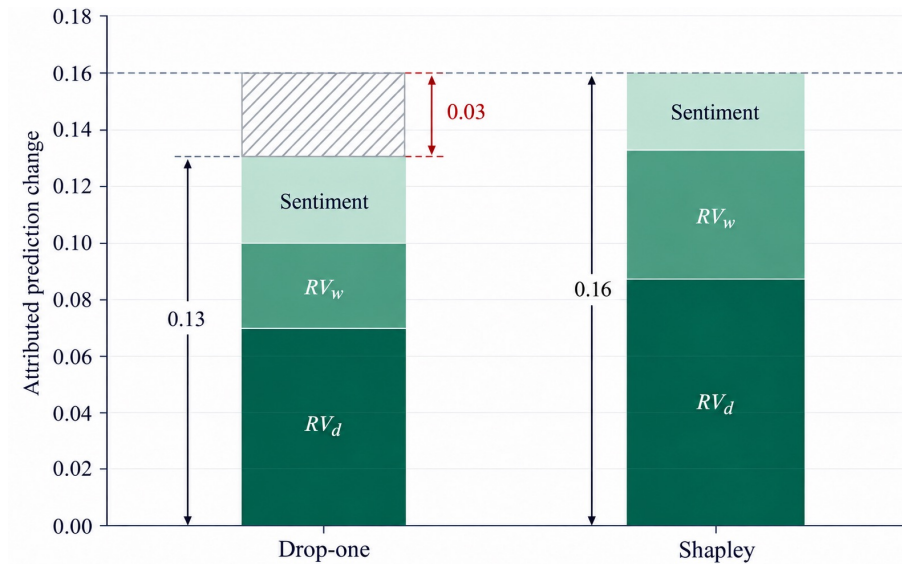


Attribution by Dropping a Single Feature

If we drop one feature at a time, the numbers do not add up.

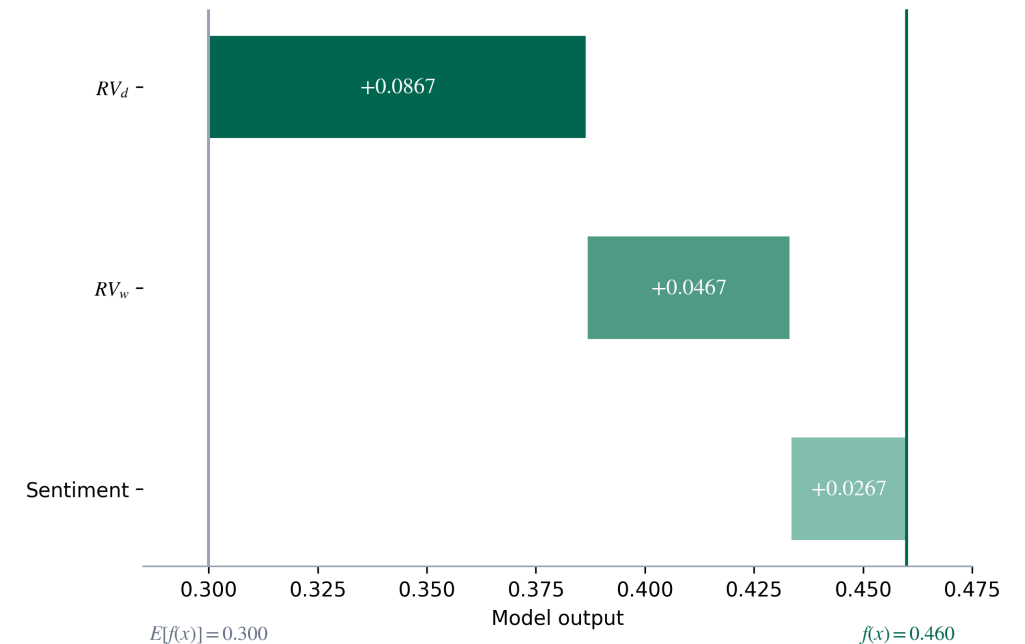
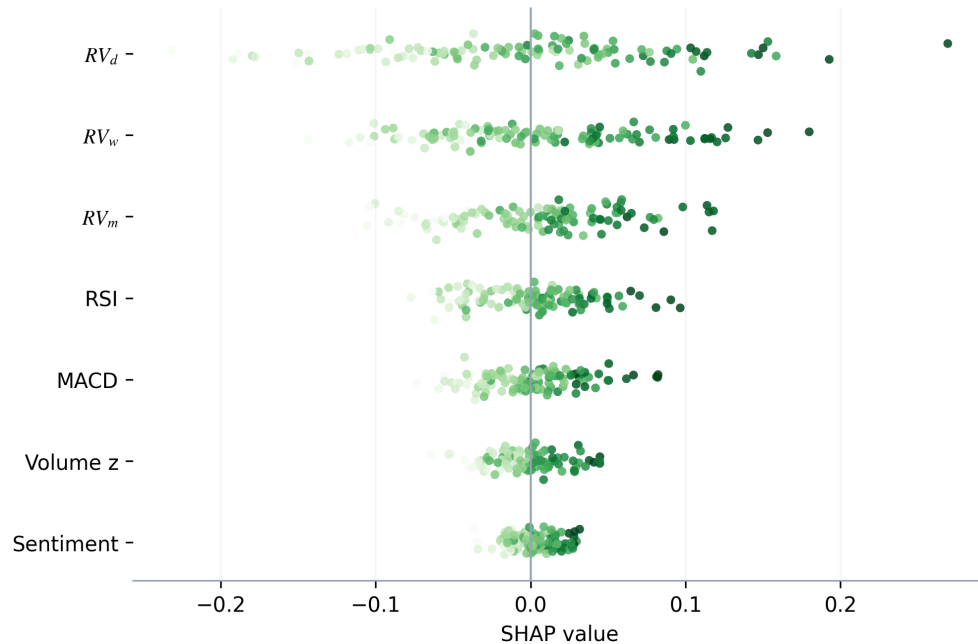
Feature	Naive: $v(\text{all}) - v(\text{all but this one})$	Value	Shapley value
RV_d	$0.16 - v(W, S) = 0.16 - 0.09$	0.07	0.0867
RV_w	$0.16 - v(D, S) = 0.16 - 0.13$	0.03	0.0467
Sentiment	$0.16 - v(D, W) = 0.16 - 0.13$	0.03	0.0267
Total		0.13	0.1600

- These add up to 0.13, but the prediction moved by 0.16, so 0.03 goes to no feature.



Reading a Beeswarm and a Waterfall

A beeswarm shows the whole model, and a waterfall shows one prediction.



- Beeswarm: features are sorted top to bottom by average effect, and each dot is one day.
- Beeswarm color: the feature's value. Dark dots on the right mean high values raise the prediction.
- Waterfall: our hand calculation, from the average prediction 0.30 to this prediction 0.46.



The Limits and the Cost of SHAP

What SHAP cannot tell us, and why the exact calculation is too expensive.

Limitation	What it means
Correlated features share credit	D and W overlap, so their credit is split
Attributions depend on the baseline	Ours was measured from 0.30; a new baseline changes every value
Attribution is not causation	SHAP describes what the model used, not what moves the market
Exact cost grows factorially	3 features give 6 orderings, 10 give 3,628,800, and 20 give about 2.4×10^{18}

TreeSHAP (Lundberg et al., 2020) is exact and fast for tree ensembles. KernelSHAP works for any model but is a sampled approximation.

SHAP shows how the model used each feature, not what moves the market.



Permutation Importance

It measures how much the model's accuracy drops when one column is shuffled.

1. Score the model on held-out rows and record the loss: the baseline.
2. Shuffle one column down the rows, leaving the others and the target alone.
3. Score again. The increase in loss is the importance of that column.
4. Restore the column, and repeat for the next one.

Original held-out rows					After permuting RV_d			
RV_d	Sent.	RSI	y		RV_d	Sent.	RSI	y
1.1	0.3	44	1	shuffle RV_d →	4.1	0.3	44	1
1.9	-0.1	52	2		1.1	-0.1	52	2
3.0	0.4	61	3		5.0	0.4	61	3
4.1	-0.2	47	4		1.9	-0.2	47	4
5.0	0.1	55	5		3.0	0.1	55	5
					y is untouched			



Permutation Importance: A Worked Example

A volatility model scored on six held-out firm-days with mean squared error.

Row	RV_d	Sent.	True y	Pred.	Sq. error	RV_d shuffled to	New pred.	New sq. error
1	1.1	0.3	1.0	1.2	0.04	4.1	2.8	3.24
2	1.9	-0.1	2.0	1.9	0.01	1.1	0.7	1.69
3	3.0	0.4	3.0	3.1	0.01	5.0	4.8	3.24
4	4.1	-0.2	4.0	3.8	0.04	1.9	2.4	2.56
5	5.0	0.1	5.0	5.2	0.04	3.0	4.1	0.81
6	6.2	0.0	6.0	5.9	0.01	6.2	5.9	0.01

Baseline MSE = 0.0250. Permuted MSE = 1.9250. Importance of RV_d = 1.900.

Doing the same for sentiment gives $0.0450 - 0.0250 = 0.020$, about 95 times smaller.



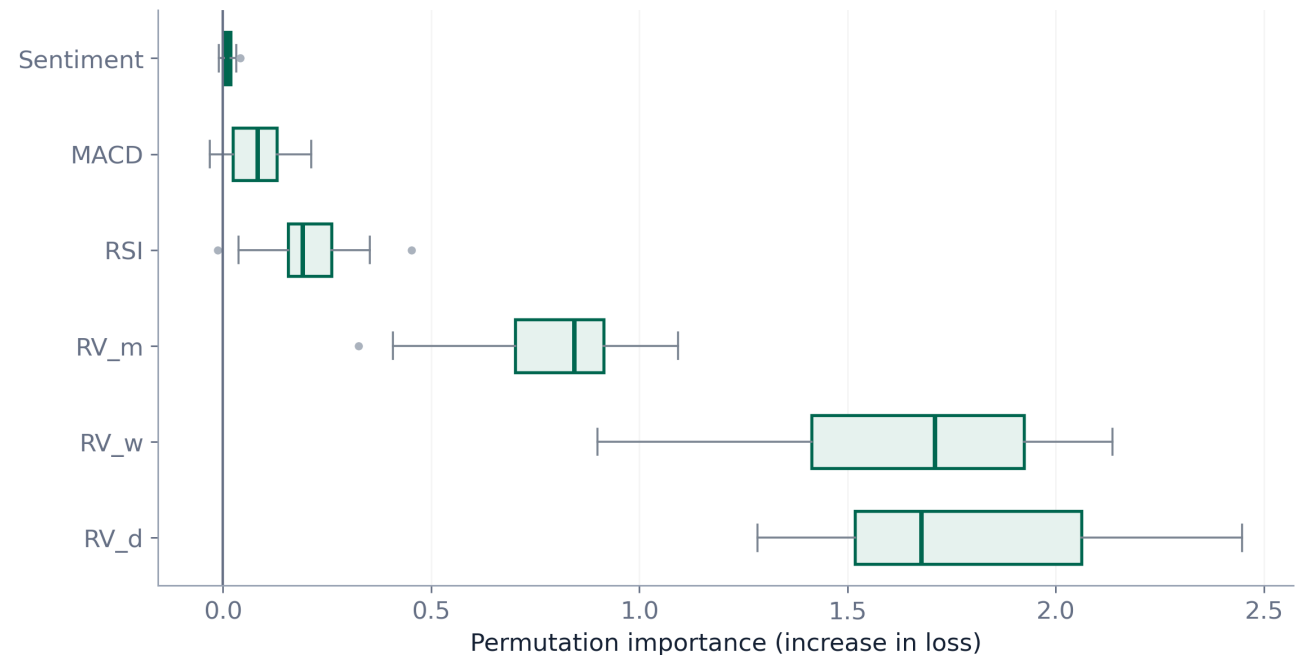
Repeated Shuffles and Sampling Error

The shuffle is random, so the importance changes from one repeat to the next.

Repeat	1	2	3	4	5	6	7	8	9	10
Sentiment importance	0.020	0.006	-0.003	0.031	0.014	-0.008	0.022	0.009	0.017	0.002

Mean = 0.0110. Standard deviation = 0.0121. Two of the ten repeats are negative.

- Repeat about thirty times and show a boxplot for each feature instead of a bar.
- For time series, shuffle blocks of consecutive rows so the autocorrelation is kept.



Correlated Features and Grouped Permutation

Hooker, Mentch and Zhou (2021), Unrestricted permutation forces extrapolation, Statistics and Computing 31.

RV_d and RV_w have a correlation of about 0.93, so shuffling only one creates unrealistic rows.

What we shuffled	Importance	Reading
RV_d alone	1.90	Looks moderate because RV_w fills in
RV_w alone	1.70	Also looks moderate, for the same reason
Sum of the two	3.60	What is often reported
$\{RV_d, RV_w\}$ together	8.40	The pair's importance as a group, over twice the sum

Shuffle correlated features together and report one importance for the group.



When Permutation Importance Is Wrong

The number can be computed correctly and still be read the wrong way.

Situation	What we see	What is really going on
Computed on training data	A memorized column looks essential	The number measures fit, not generalization
A duplicated column	Both copies score near zero	Either one alone would do the job
Time series shuffled row by row	A lagged feature looks huge	The shuffle also broke the autocorrelation
Read as a cause	This feature drives volatility	The model relies on it; the market may not

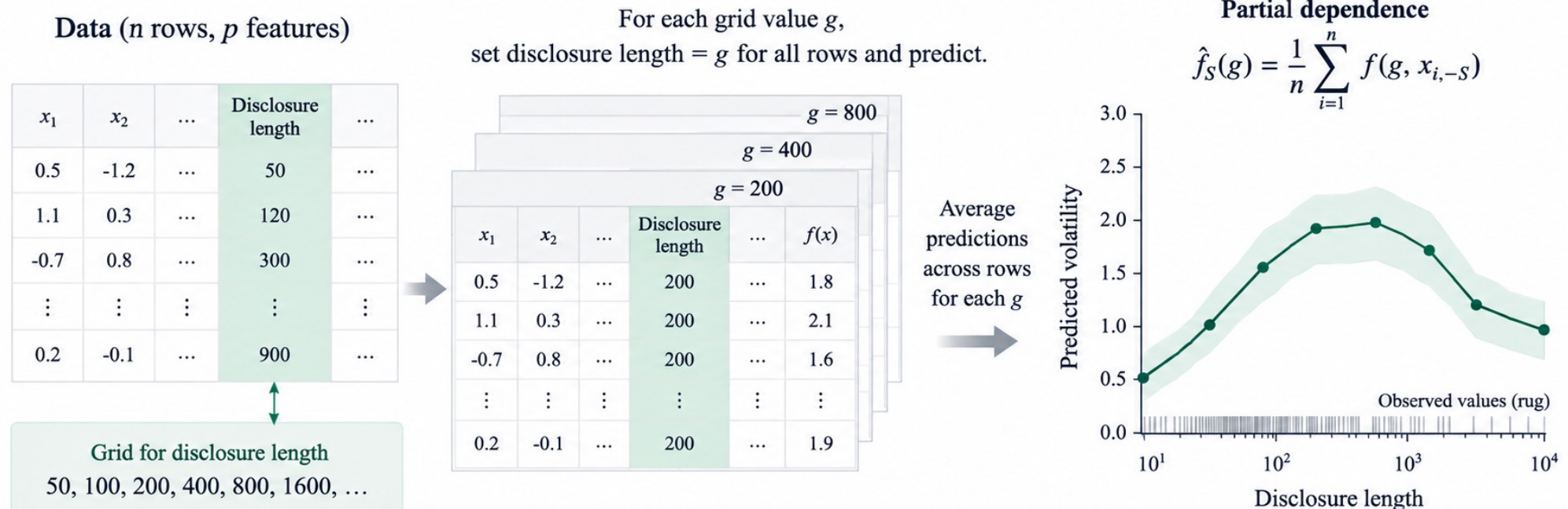
Importance describes the model on this sample, not the market.



Partial Dependence

Importance says which features the model uses, and partial dependence shows how it uses them.

1. Choose a feature and a grid of values for it.
2. Overwrite that feature in every row with one grid value, and predict.
3. Average the predictions, and plot the averages against the grid.

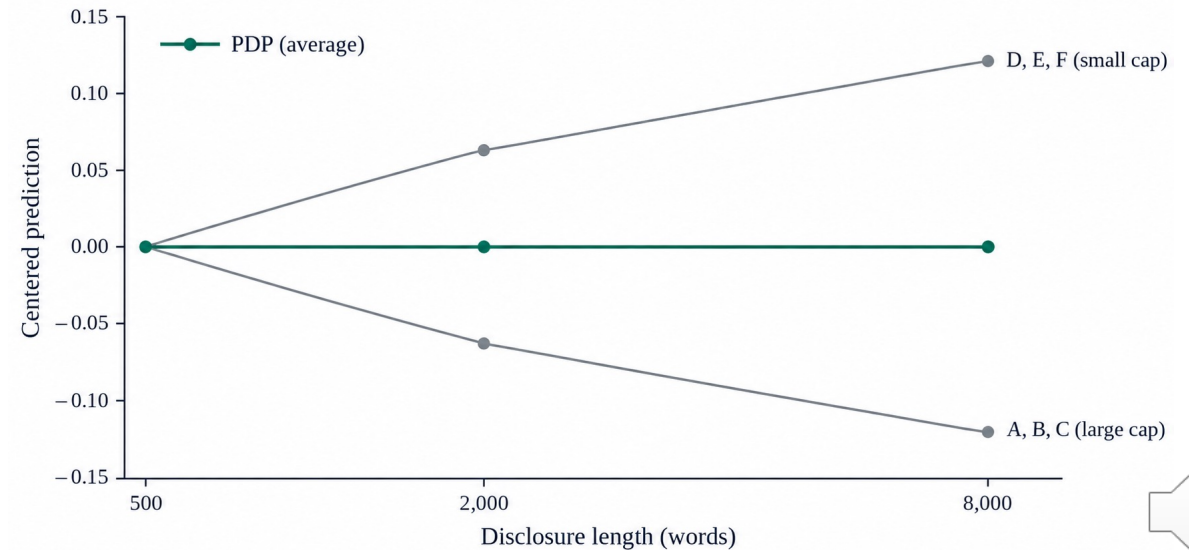


The Flat Partial Dependence Curve

Predicted volatility for six firms as disclosure length is set to 500, 2,000 and 8,000 words.

Firm	500 words	2,000 words	8,000 words	Change from 500 to 8,000
A, B, C (large cap)	0.30 / 0.28 / 0.32	0.24 / 0.22 / 0.26	0.18 / 0.16 / 0.20	-0.12 each
D, E, F (small cap)	0.20 / 0.18 / 0.22	0.26 / 0.24 / 0.28	0.32 / 0.30 / 0.34	+0.12 each
PDP (column average)	0.250	0.250	0.250	0.000

- The average curve is flat, but no single firm is flat: the two groups move in opposite directions and cancel.
- An individual conditional expectation (ICE) plot draws one line per firm and shows both groups.



When a Partial Dependence Curve Is Wrong

Partial dependence runs into the same problems as permutation importance.

Situation	Why the curve misleads
Correlated features	Setting RV_d to 6.2 with RV_w at 1.3 makes an unrealistic row
Few observations at the ends	The curve there rests on predictions without data
Effects that differ by group	Opposite effects in different subgroups cancel out
Read as what would happen	It shows model output, not what a longer filing would cause

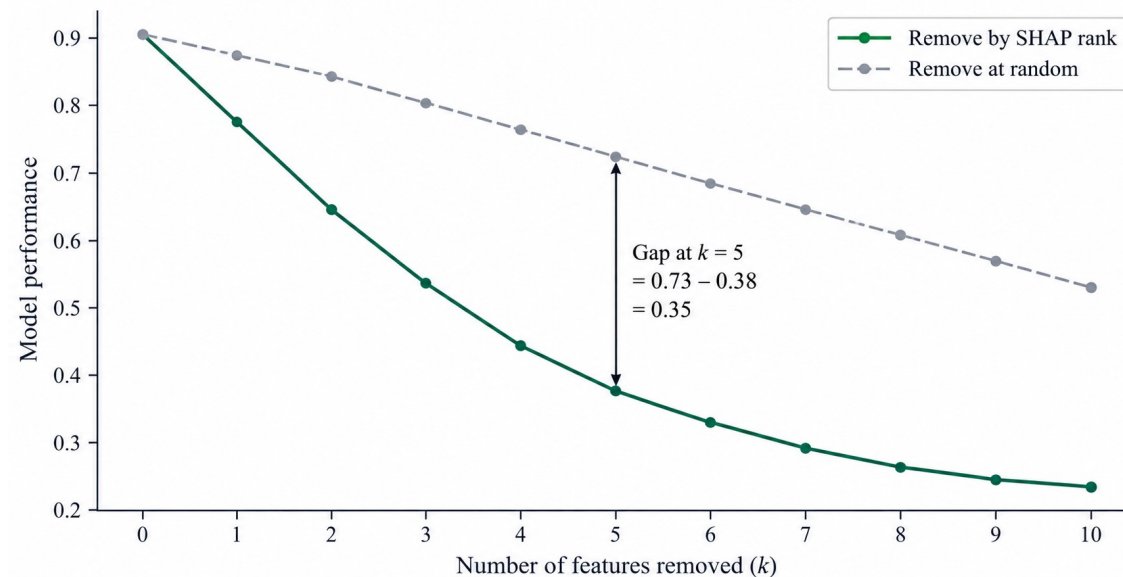
- For correlated features, use Accumulated Local Effects (Apley and Zhu, 2020), which only compares nearby values in the data.
- Above a 0.7 correlation, read a partial dependence curve as a rough sketch.



Validating an Explanation

Criteria from Samek et al. (2017), Alvarez-Melis and Jaakkola (2018), and Jacovi and Goldberg (2020).

Check	Procedure	Sign of failure
Faithfulness	Remove the top-k attributed features and measure the drop	No bigger drop than removing k at random
Stability	Rerun with slightly changed inputs or new seeds	The ranking changes a lot
Plausibility	Ask whether the top features make sense to an expert	No sensible reason explains the top feature



Two Ways to Explain a Text Model

A text model can be explained through the words it reads or through reasoning it writes out.

"The Company reversed \$42 million of the impairment charge recorded in the prior year, and settled the outstanding litigation at no cost."

	Token attribution	Written reasoning
What it gives	A score for each word	Sentences giving a judgment and its evidence
Example	'reversed' pushed the label to positive	Bullish: most posts cite the new contract
Methods	Attention, token-level SHAP	Prompting a language model
What it adds	A view inside one model	An explanation, and often better accuracy

The first treats words as features; the second reads the text the way an analyst would.

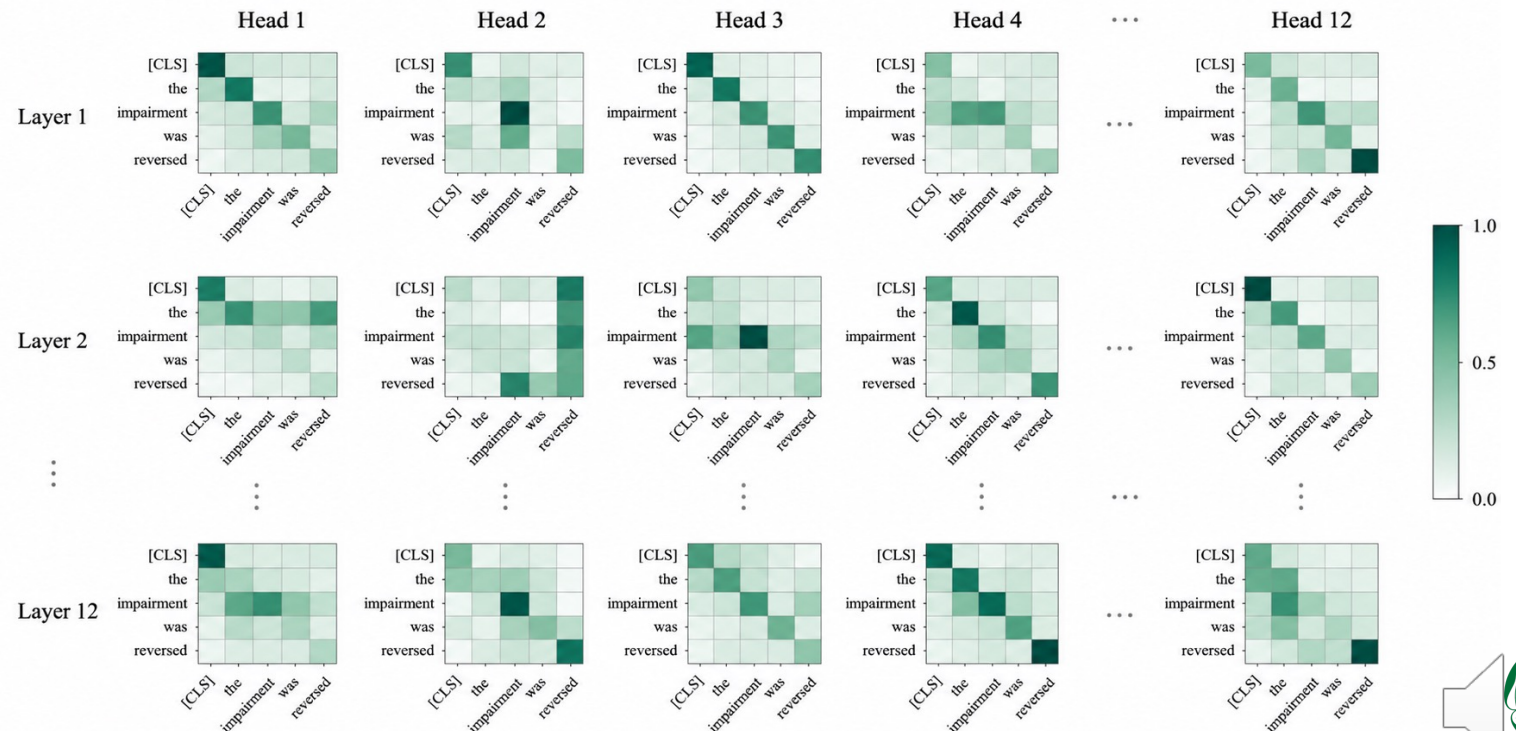


The Attention Mechanism

Each token's new representation is a weighted average of all the tokens in the sentence.

Token	[CLS]	the	impairment	was	reversed	Sum
Attention weight	0.02	0.05	0.60	0.03	0.30	1.00

- The weights are non-negative and sum to one because they come from a softmax.
- Summing to one is built in, so a large weight does not mean a large effect on the output.



Attention Weights as Explanation

Jain and Wallace (2019), Attention is not Explanation, NAACL-HLT. Approx. 2,700 citations.

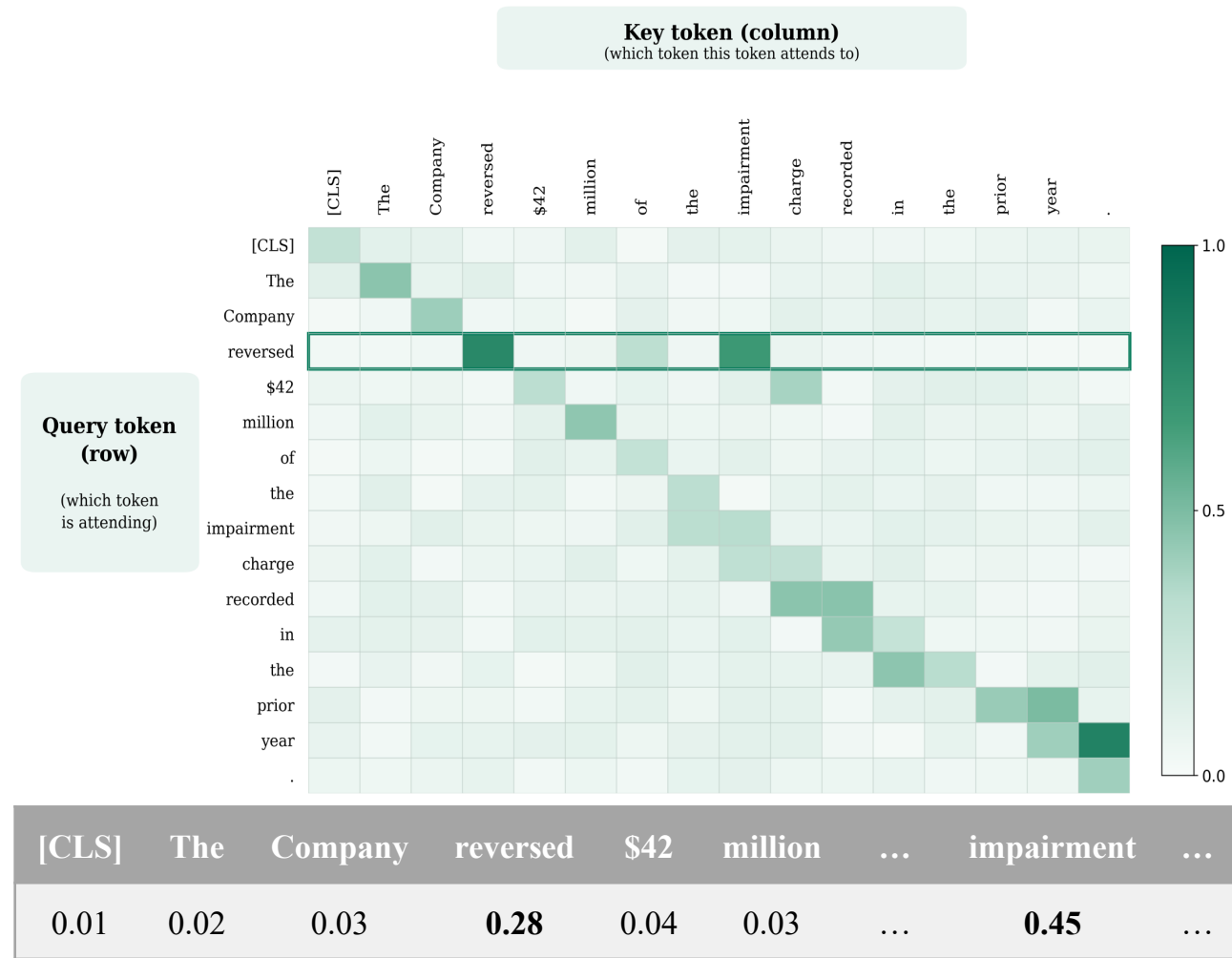
	[CLS]	the	impairment	was	reversed	p(positive)
Original attention	0.02	0.05	0.60	0.03	0.30	0.88
Adversarial attention	0.45	0.40	0.02	0.11	0.02	0.88
Overlap (smaller of the two)	0.02	0.05	0.02	0.03	0.02	sum 0.14

- The two attention patterns overlap by only 14 percent, yet the model gives the same answer.

Attention weights are useful for exploring, but not as evidence of why the model decided.

Reading an Attention Heatmap

The attention weights of one head over a sentence that BERT scored.



How to read it

1. Choose a row

- Each row represents one **query token**. Read across the row to see where its attention goes.

2. Compare the weights

- Darker cells indicate **larger attention weights**.
For “**reversed**,” the largest weights are:
impairment 0.45, reversed 0.28

3. Interpret with care

- Weights in each row sum to 1.
High attention $\neq$ high influence on the final prediction.



A Language Model as the Writer of the Explanation

A decoder model predicts the next token, and instruction tuning teaches it to follow requests.

Now the model writes out in words what a result means, rather than assigning a label.

Task	Input	Output
Explain a SHAP result	A SHAP table for one prediction	A paragraph a portfolio manager can read
Give a reason	One document and a label	The model's reason for that label
Build a column	Many documents and a fixed question	One record per document, checkable by hand

Asking a Model Why It Chose a Label

We score the sentence from the sentiment lecture again, this time with a reason.

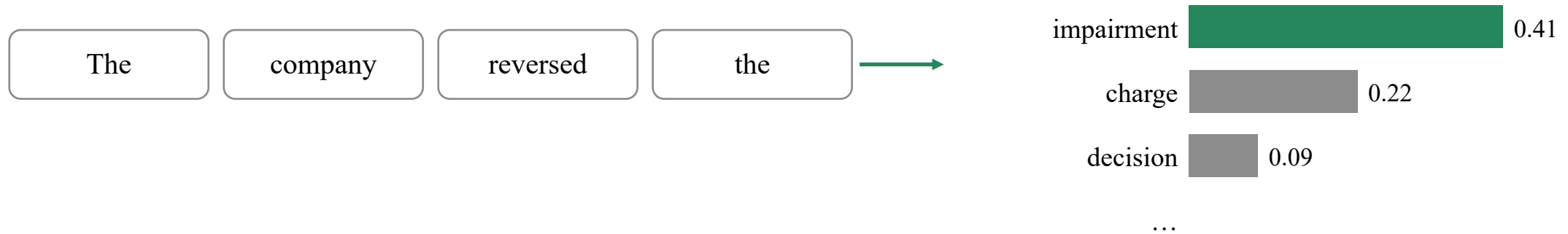
Field	Value
Sentence	The Company reversed \$42 million of the impairment charge recorded in the prior year.
Label	positive
Stated reason	Reversing a charge recorded earlier adds income in the current period
What can be checked	The words are in the sentence; the model's real process is not

The model gives a convincing reason even when the label is wrong.



Next-Token Prediction

A decoder writes text one token at a time, each time choosing from probabilities over its vocabulary.



- Training: predict the next token at every position of a very large body of text.
- Generation: pick a token, append it to the input, and predict the next one again.
- Everything a language model writes, including an explanation, is built from these single steps.



The Causal Mask

In a decoder, each token can attend only to itself and to the tokens before it.

	The	firm	reversed	the	charge
The					
firm					
reversed					
the					
charge					

- Rows are the token being computed; columns are the tokens it may look at.
- The upper triangle is masked, so no token can see the tokens that come after it.
- A token the model has just written becomes visible to every token after it.

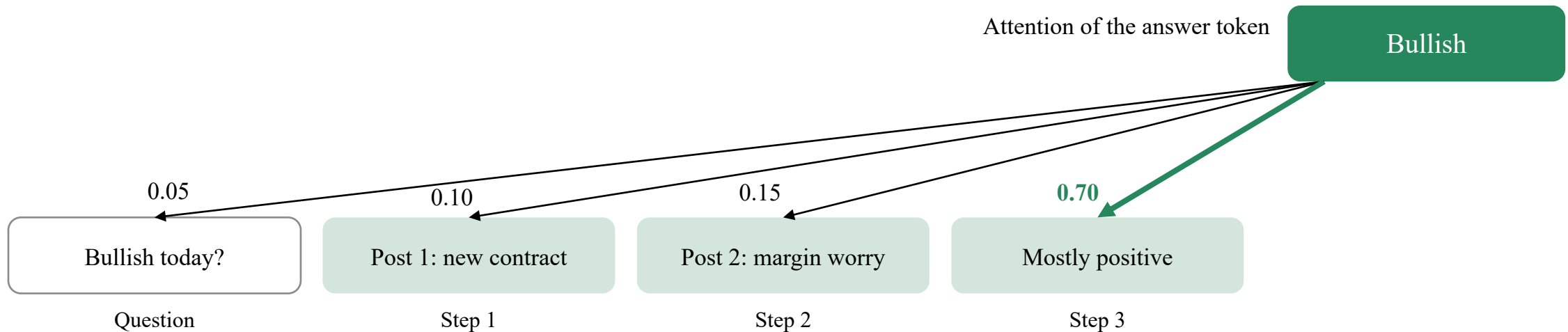
So whatever the model writes down, including its own reasoning, becomes input for what it writes next.



Reasoning in a Language Model

A language model can reason by writing out intermediate steps before its answer.

- Each new token is predicted from all earlier tokens, including the ones the model just wrote.
- Each token gets about the same computation, so writing out steps spreads a hard problem over more tokens.
- Wei et al. (2022): asking for step-by-step reasoning improved accuracy on multi-step problems.
- Reasoning models such as OpenAI o1 and DeepSeek-R1 are trained to reason at length before answering.



Reasoning as Explanation and as Performance

Asked to reason, a model returns its evidence and often a more accurate answer.

Field	Model output
Question	Is today's board consensus on the stock bullish, and why?
Evidence	3 of 4 posts cite a new supply contract; 1 worries about margins
Reasoning	Most posts point to new revenue, and the one concern is minor
Answer	Bullish

The evidence can be checked against the posts, but the reasoning may not be why the model answered.



Chain of Thought and Faithfulness

The reasoning a model writes out may not be the real reason for its answer.

- Why it is attractive: the model explains step by step in plain language.
- Turpin et al. (2023): a hidden cue changed the answers, and the reasoning never mentioned it.
- Chen et al. (2025): reasoning models admitted using a hint in fewer than half of the cases.

Original order

Which option best follows the rule?

A. option one

B. option two

C. option three

D. option four

Answer: B

Reasoning: coherent step-by-step justification...

The reordering cue is never mentioned.



Quietly reordered

Which option best follows the rule?

D. option four

B. option two

C. option three

A. option one

Answer: D

Reasoning: coherent step-by-step justification...

The reordering cue is never mentioned.



Prompting for an Explanation That Can Be Checked

To have a language model explain a SHAP result, write the prompt so the answer can be checked.

Part of the prompt	In our prompt	Why it helps
Role and reader	“You write a risk note for a portfolio manager.”	Keeps it a short report, not market commentary
The numbers	“Baseline 0.30. $RV_d + 0.087$, $RV_w + 0.047$, Sentiment +0.027. Prediction 0.46.”	The model copies numbers instead of guessing them
A ban	“Do not mention any cause that is not in the table.”	Blocks made-up causes such as interest rates
A format	“Three sentences. Each names one feature and its value.”	Missing or extra claims are easy to spot
One example	One sample sentence in the style you want	Shows the style faster than describing it

Every number in the answer should appear in the prompt. If it does not, the model made it up.



The Risk in a Narrative Explanation

A paragraph can read smoothly and still be wrong.

What goes wrong	Example	How to check
A made-up cause	Rates rose, so volatility rose	No rate feature was in the table
A number not in the input	Volatility is up 40 percent	Recompute it from the inputs
Vague but confident wording	Likely driven by	Ask which attribution supports it
Causal language	This feature drives volatility	Rewrite as 'the model relies on it'

Even when the SHAP values are right, the paragraph about them can be wrong.



Tokenization: How a Language Model Reads Text

A language model reads token IDs looked up in a table, never the letters or digits themselves.

Token	ID (illustrative)	Vector looked up from the table
'stock'	4612	0.21, -0.37, 0.08, ...
'487'	22148	-0.15, 0.44, 0.30, ...
' reversed'	28537	0.05, 0.12, -0.61, ...

Well-known failures in arithmetic, spelling and letter counting all start at this step.



Arithmetic Errors

Doubling a number does not double its vector, because each token's vector was learned separately.

Written as	BPE tokens	What the model sees
4871	'487' + '1'	Two rows of the table, not a quantity
3562	'356' + '2'	Split the same way
4,871	'4' + ',' + '871'	A comma changes the split

$487 \times 2 = 974$, but the vector for '974' is not twice the vector for '487'. Multiplication is not built into the vectors.

So calculations should go to a tool, such as Python.



Spelling and Letter-Counting Errors

In the well-known 'strawberry' problem, the letters are hidden inside tokens.

Task	Example	Tokens (illustrative)	Why it is hard
Reversing a word	'stock' → 'kcots'	'stock' vs 'k' + 'cots'	One familiar token becomes several unfamiliar ones
Counting letters	The r's in 'strawberry'	'str' + 'aw' + 'berry'	The r's are hidden inside 'str' and 'berry'
Sorting by spelling	Tickers in alphabetical order	'AAPL', 'NV' + 'DA', 'MS' + 'FT'	The model compares tokens, not letters

A person reads letters, but the model reads tokens, so letter-level tasks belong in code.



Working Around Tokenization

Knowing that a model thinks in tokens tells us what to ask it for and what to compute ourselves.

Task	For the model	What to do
Numbers and calculations	Weak	Compute in Python and paste the results in
Letters, spelling, sorting	Weak	Do it in code
Quoting the source text	Strong: it copies tokens	Ask for exact quotes as evidence
Judgment in words	Strong, but may not be faithful	Check the quoted evidence

Let the model read and write, and let code count and calculate.

